Министерство науки и высшего образования РФ
ФГАОУ ВПО
Национальный исследовательский технологический университет
«МИСИС»

Институт информационных технологий и компьютерных наук (ИТКН)

Кафедра инфокоммуникационных технологий (ИКТ)

Отчет по лабораторной работе № 4
по дисциплине «Объектно-ориентированное программирование»

на тему «Разработка классов в C#»

Выполнил:
студент группы БИВТ-25-1
Кирюшин А. А.
Проверил:
доцент каф. ИКТ
Стучилин В.В.

Москва, 2026

1. ЦЕЛЬ РАБОТЫ

Приобрести навыки работы с классами на языке программирования Си#.

2. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

В рамках лабораторной работы были решены следующие задачи:

- Создать класс Currency с полями код и курс валюты, методами конвертации
- Создать класс Account с полем баланс, методами зачисления/снятия средств
- Реализовать консольное приложение для работы со счетом в трех валютах (RUR, INR, USD)
- Обеспечить инкапсуляцию данных (private поля, public методы)
- Реализовать автоматическую конвертацию валют при операциях

5. ЛИСТИНГ ПРОГРАММЫ

Класс Currency:

Решение:

```
namespace BankAccount
{
    1 reference
    public class Currency
    {
        3 references
        private string code;
        5 references
        private double exchangeRate;

        0 references
        public string Code
        {
            get { return code; }
        }

        0 references
        public double ExchangeRate
        {
            get { return exchangeRate; }
        }

        0 references
        public Currency(string code, double exchangeRate)
        {
            this.code = code;
            this.exchangeRate = exchangeRate;
        }

        0 references
        public double ConvertToBase(double amount)
        {
            return amount * exchangeRate;
        }

        0 references
        public double ConvertFromBase(double amount)
        {
            return amount / exchangeRate;
        }

        0 references
        public override string ToString()
        {
            return $"{code} (курс: {exchangeRate:F2} RUR)";
        }
    }
}
```

Класс Account:

Решение:

```
namespace BankAccount
{
    1 reference
    public class Account
    {
        6 references
        private double balance;

        0 references
        public Account()
        {
            balance = 0.0;
        }

        0 references
        public double GetBalance()
        {
            return balance;
        }

        0 references
        public double GetBalanceInCurrency(Currency currency)
        {
            return currency.ConvertFromBase(balance);
        }

        0 references
        public bool Deposit(double amount, Currency currency)
        {
            if (amount <= 0)
            {
                return false;
            }

            double amountInBase = currency.ConvertToBase(amount);
            balance += amountInBase;
            return true;
        }

        0 references
        public bool Withdraw(double amount, Currency currency)
        {
            if (amount <= 0)
            {
                return false;
            }

            double amountInBase = currency.ConvertToBase(amount);

```

```
43                  if (balance < amountInBase)
44                  {
45                      return false;
46                  }
47
48                  balance -= amountInBase;
49                  return true;
50              }
51          }
52      }
53
```

Графический интерфейс:

Решение:

```
namespace GeometryGUI.Views
{
    public partial class MainWindow : Window
    {
        private List<GeometricShape> shapes = new
List<GeometricShape>();
        private GeometricShape? selectedShape = null;
        private Random random = new Random();
        private string[] colors = { "Red", "Blue", "Green",
"Orange", "Purple", "Brown", "Pink", "Cyan" };

        public MainWindow()
        {
            InitializeComponent();
        }

        private void OnCreateRandomClick(object? sender,
RoutedEventArgs e)
        {
            double canvasWidth =
DrawingCanvas.Bounds.Width;
            double canvasHeight =
DrawingCanvas.Bounds.Height;

            if (canvasWidth <= 0 || canvasHeight <= 0)
            {
                canvasWidth = 600;
                canvasHeight = 600;
            }

            int shapeType = random.Next(0, 3);
            double centerX = random.Next(100,
(int)canvasWidth - 100);
            double centerY = random.Next(100,
(int)canvasHeight - 100);
            string color =
colors[random.Next(colors.Length)];

            GeometricShape shape;
            switch (shapeType)
            {
```

```
                case 0:
                    double side = random.Next(30, 100);
                    shape = new Square(centerX, centerY,
side, color);
                    break;
                case 1:
                    double diameter = random.Next(40, 120);
                    shape = new Circle(centerX, centerY,
diameter, color);
                    break;
                default:
                    double height = random.Next(40, 100);
                    shape = new Triangle(centerX, centerY,
height, color);
                    break;
            }

            shapes.Add(shape);
            DrawShape(shape);
            StatusLabel.Text = $"Создана фигура:
{shape.GetType().Name}";
        }

        private void DrawShape(GeometricShape shape)
        {
            Shape visual;
            IBrush brush = new
SolidColorBrush(ParseColor(shape.Color));
            IBrush stroke = Brushes.Black;

            if (shape is Square square)
            {
                visual = new Rectangle
                {
                    Width = square.Side,
                    Height = square.Side,
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2
                };
                Canvas.SetLeft(visual, shape.CenterX -
square.Side / 2);
                Canvas.SetTop(visual, shape.CenterY -
square.Side / 2);
            }
            else if (shape is Circle circle)
```

```
            {
                visual = new Ellipse
                {
                    Width = circle.Diameter,
                    Height = circle.Diameter,
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2
                };
                Canvas.SetLeft(visual, shape.CenterX -
circle.Diameter / 2);
                Canvas.SetTop(visual, shape.CenterY -
circle.Diameter / 2);
            }
            else if (shape is Triangle triangle)
            {
                double halfBase = triangle.Height / 2;
                Polygon polygon = new Polygon
                {
                    Fill = brush,
                    Stroke = stroke,
                    StrokeThickness = 2,
                    Points = new
Avalonia.Collections.AvaloniaList<Avalonia.Point>
                    {
                        new Avalonia.Point(0,
-triangle.Height * 2.0 / 3.0),
                        new Avalonia.Point(-halfBase,
triangle.Height / 3.0),
                        new Avalonia.Point(halfBase,
triangle.Height / 3.0)
                    }
                };
                visual = polygon;
                Canvas.SetLeft(visual, shape.CenterX);
                Canvas.SetTop(visual, shape.CenterY);
            }
            else
            {
                return;
            }

            visual.Tag = shape;
            DrawingCanvas.Children.Add(visual);
        }
```

```
        private void OnCanvasPointerPressed(object? sender,
PointerPressedEventArgs e)
        {
            var point = e.GetPosition(DrawingCanvas);
            double x = point.X;
            double y = point.Y;

            selectedShape = null;
            foreach (var shape in shapes)
            {
                if (shape.ContainsPoint(x, y))
                {
                    selectedShape = shape;
                    break;
                }
            }

            if (selectedShape != null)
            {
                ShapeInfoLabel.Text = $"Тип:
{selectedShape.GetType().Name}\n" +
                                        $"Центр:
({selectedShape.CenterX:F1}, {selectedShape.CenterY:F1})\n"
+
                                        $"Цвет:
{selectedShape.Color}\n" +
                                        $"Площадь:
{selectedShape.CalculateArea():F2}";
                selectedShape.SelectShape();
            }
            else
            {
                ShapeInfoLabel.Text = "Выберите фигуру на
холсте";
            }
        }

        private void OnCheckPointClick(object? sender,
RoutedEventArgs e)
        {
            if (!double.TryParse(PointXTextBox.Text, out
double x) ||
                !double.TryParse(PointYTextBox.Text, out
double y))
            {
```

```
                PointResultLabel.Text = "Ошибка: введите
корректные координаты";
                PointResultLabel.Foreground = Brushes.Red;
                return;
            }

            if (selectedShape == null)
            {
                PointResultLabel.Text = "Выберите фигуру на
холсте";
                PointResultLabel.Foreground =
Brushes.Orange;
                return;
            }

            bool inside = selectedShape.ContainsPoint(x,
y);
            PointResultLabel.Text = inside
                ? $"Точка ({x:F1}, {y:F1}) ВНУТРИ выбранной
фигуры"
                : $"Точка ({x:F1}, {y:F1}) ВНЕ выбранной
фигуры";
            PointResultLabel.Foreground = inside ?
Brushes.Green : Brushes.Red;
        }

        private void OnClearAllClick(object? sender,
RoutedEventArgs e)
        {
            shapes.Clear();
            DrawingCanvas.Children.Clear();
            selectedShape = null;
            ShapeInfoLabel.Text = "Выберите фигуру на
холсте";
            StatusLabel.Text = "Холст очищен";
        }

        private void OnSaveClick(object? sender,
RoutedEventArgs e)
        {
            try
            {
                string filePath = "shapes.txt";
                using (StreamWriter writer = new
StreamWriter(filePath))
                {
```

```
                    foreach (var shape in shapes)
                    {
                        if (shape is Square square)
                        {
                            writer.WriteLine($"Square|
{square.CenterX}|{square.CenterY}|{square.Side}|
{square.Color}");
                        }
                        else if (shape is Circle circle)
                        {
                            writer.WriteLine($"Circle|
{circle.CenterX}|{circle.CenterY}|{circle.Diameter}|
{circle.Color}");
                        }
                        else if (shape is Triangle
triangle)
                        {
                            writer.WriteLine($"Triangle|
{triangle.CenterX}|{triangle.CenterY}|{triangle.Height}|
{triangle.Color}");
                        }
                    }
                }
                StatusLabel.Text = $"Сохранено
{shapes.Count} фигур в файл {filePath}";
                StatusLabel.Foreground = Brushes.Green;
            }
            catch (Exception ex)
            {
                StatusLabel.Text = $"Ошибка сохранения:
{ex.Message}";
                StatusLabel.Foreground = Brushes.Red;
            }
        }

        private void OnLoadClick(object? sender,
RoutedEventArgs e)
        {
            try
            {
                string filePath = "shapes.txt";
                if (!File.Exists(filePath))
                {
                    StatusLabel.Text = "Файл shapes.txt не
найден";
```

```
                    StatusLabel.Foreground =
Brushes.Orange;
                    return;
                }

                shapes.Clear();
                DrawingCanvas.Children.Clear();

                string[] lines =
File.ReadAllLines(filePath);
                foreach (string line in lines)
                {
                    string[] parts = line.Split('|');
                    if (parts.Length < 5) continue;

                    string type = parts[0];
                    double centerX =
double.Parse(parts[1]);
                    double centerY =
double.Parse(parts[2]);
                    double size = double.Parse(parts[3]);
                    string color = parts[4];

                    GeometricShape shape;
                    switch (type)
                    {
                        case "Square":
                            shape = new Square(centerX,
centerY, size, color);
                            break;
                        case "Circle":
                            shape = new Circle(centerX,
centerY, size, color);
                            break;
                        case "Triangle":
                            shape = new Triangle(centerX,
centerY, size, color);
                            break;
                        default:
                            continue;
                    }

                    shapes.Add(shape);
                    DrawShape(shape);
                }
```

```
                StatusLabel.Text = $"Загружено
{shapes.Count} фигур из файла {filePath}";
                StatusLabel.Foreground = Brushes.Green;
            }
            catch (Exception ex)
            {
                StatusLabel.Text = $"Ошибка загрузки:
{ex.Message}";
                StatusLabel.Foreground = Brushes.Red;
            }
        }

        private Color ParseColor(string colorName)
        {
            return colorName.ToLower() switch
            {
                "red" => Colors.Red,
                "blue" => Colors.Blue,
                "green" => Colors.Green,
                "orange" => Colors.Orange,
                "purple" => Colors.Purple,
                "brown" => Colors.Brown,
                "pink" => Colors.Pink,
                "cyan" => Colors.Cyan,
                _ => Colors.Gray
            };
        }
    }
}
```

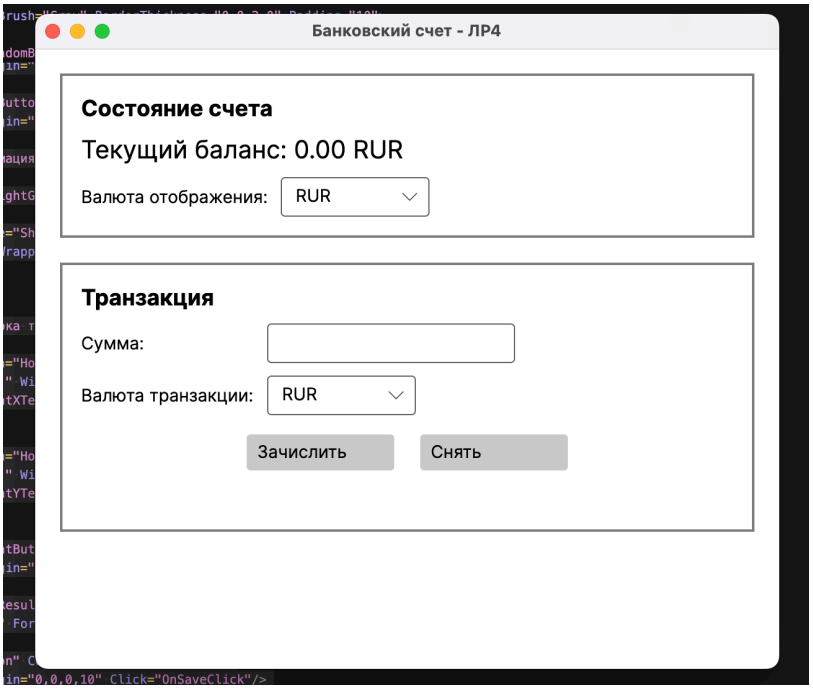
Интерфейс:

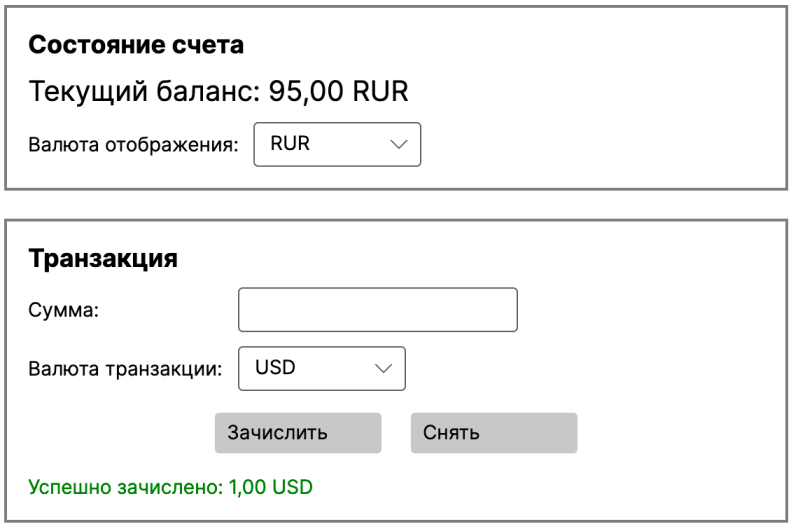
```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/
xaml"
        x:Class="GeometryGUI.Views.MainWindow"
        Title="Геометрические фигуры - ЛР5"
        Width="900" Height="700">

    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="250"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- Левая панель управления -->
```

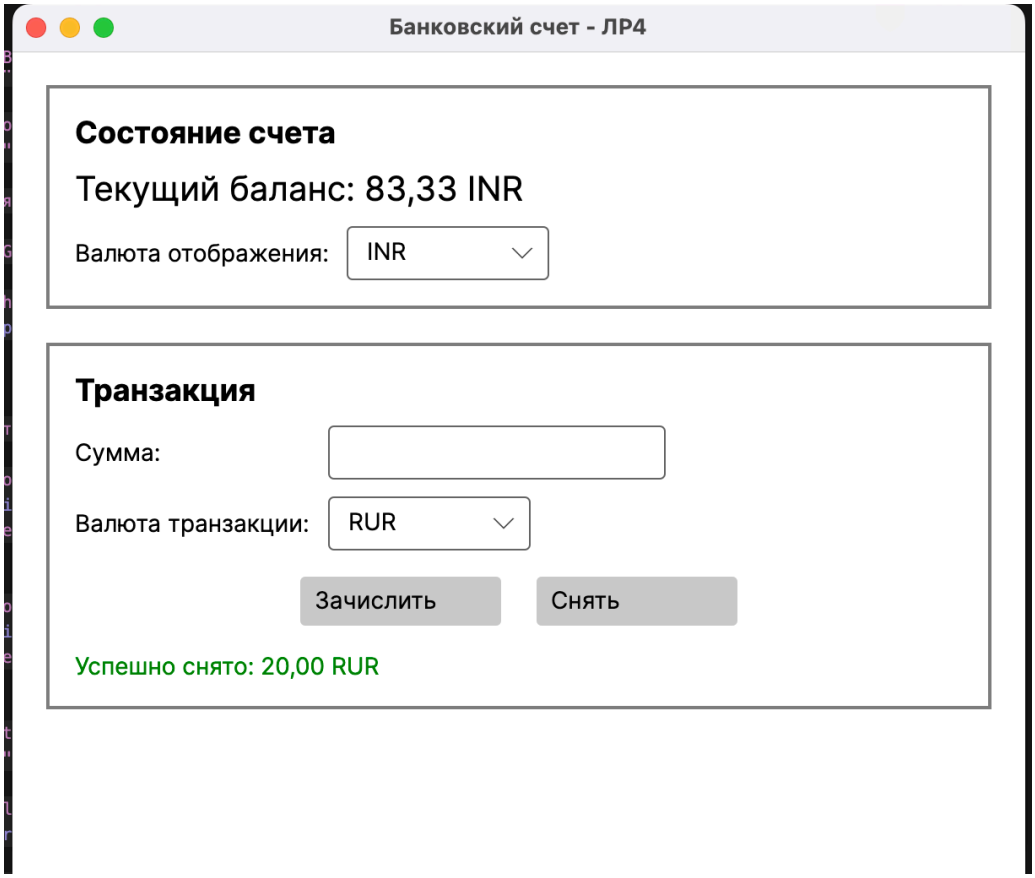
```
        <Border Grid.Column="0" BorderBrush="Gray"
BorderThickness="0,0,2,0" Padding="10">
            <StackPanel>
                <TextBlock Text="Управление фигурами"
FontSize="18" FontWeight="Bold" Margin="0,0,0,15"/>

                <Button Name="CreateRandomButton"
Content="Создать случайную фигуру"
                        Width="220" Margin="0,0,0,10"
Click="OnCreateRandomClick"/>

                <Button Name="ClearAllButton"
Content="Очистить холст"
                        Width="220" Margin="0,0,0,20"
Click="OnClearAllClick"/>

                <TextBlock Text="Информация о фигуре"
FontSize="16" FontWeight="Bold" Margin="0,0,0,10"/>

                <Border BorderBrush="LightGray"
BorderThickness="1" Padding="10" Margin="0,0,0,10">
                    <StackPanel>
                        <TextBlock Name="ShapeInfoLabel"
Text="Выберите фигуру на холсте"
                                   TextWrapping="Wrap"
FontSize="12"/>
                    </StackPanel>
                </Border>

                <TextBlock Text="Проверка точки"
FontSize="16" FontWeight="Bold" Margin="0,10,0,10"/>

                <StackPanel Orientation="Horizontal"
Margin="0,0,0,5">
                    <TextBlock Text="X:" Width="30"
VerticalAlignment="Center"/>
                    <TextBox Name="PointXTextBox"
Width="80"/>
                </StackPanel>

                <StackPanel Orientation="Horizontal"
Margin="0,0,0,10">
                    <TextBlock Text="Y:" Width="30"
VerticalAlignment="Center"/>
                    <TextBox Name="PointYTextBox"
Width="80"/>
```

```
                </StackPanel>

                <Button Name="CheckPointButton"
Content="Проверить точку"
                        Width="220" Margin="0,0,0,10"
Click="OnCheckPointClick"/>

                <TextBlock Name="PointResultLabel" Text=""
TextWrapping="Wrap"
                           FontSize="12"
Foreground="DarkBlue" Margin="0,5,0,15"/>

                <Button Name="SaveButton"
Content="Сохранить в файл"
                        Width="220" Margin="0,0,0,10"
Click="OnSaveClick"/>

                <Button Name="LoadButton"
Content="Загрузить из файла"
                        Width="220" Click="OnLoadClick"/>

                <TextBlock Name="StatusLabel" Text=""
TextWrapping="Wrap"
                           FontSize="11" Foreground="Green"
Margin="0,10,0,0"/>
            </StackPanel>
        </Border>

        <!-- Правая панель с Canvas -->
        <Border Grid.Column="1" Background="White">
            <Canvas Name="DrawingCanvas" Background="White"
                    PointerPressed="OnCanvasPointerPressed"/
>
        </Border>
    </Grid>

</Window>
```

Выполнение:


Банковский счет - ЛР4
Состояние счета
Текущий баланс: 0.00 RUR
Валюта отображения:
RUR
Транзакция
Сумма:
Валюта транзакции:
RUR
Зачислить
Снять


Состояние счета
Текущий баланс: 95,00 RUR
Валюта отображения:
RUR
Транзакция
Сумма:
Валюта транзакции:
USD
Зачислить
Снять
Успешно зачислено: 1,00 USD


Банковский счет - ЛР4
Состояние счета
Текущий баланс: 83,33 INR
Валюта отображения:
INR
Транзакция
Сумма:
Валюта транзакции:
RUR
Зачислить
Снять
Успешно снято: 20,00 RUR

7. ВЫВОД

В ходе выполнения лабораторной работы я успешно освоил базовые принципы объектно-ориентированного программирования на языке C#.
Получил практический опыт создания классов с приватными полями и публичными методами, что обеспечивает инкапсуляцию данных и защиту от некорректного использования. Научился правильно объявлять свойства классов с методами доступа get/set, использовать конструкторы для инициализации объектов с заданными параметрами.

Реализовал два взаимодействующих класса: Currency (Валюта) и Account (Счет). Класс Currency инкапсулирует логику работы с валютой и методы конвертации курсов, предоставляя простой интерфейс для преобразования сумм между валютами. Класс Account использует объекты Currency для выполнения операций зачисления и снятия средств в разных валютах, при этом внутреннее хранение баланса происходит в единой базовой валюте.
Научился создавать объекты классов с помощью ключевого слова new, работать с массивами объектов, понял разницу между классом как типом данных и объектом как конкретным экземпляром этого типа. Освоил работу со ссылочными типами и понял, что переменные классов хранят ссылки на объекты в памяти.

Применил принципы хорошего проектирования классов: разделение ответственности (каждый класс решает свою задачу), сокрытие деталей реализации через приватные поля, предоставление контролируемого доступа к данным через публичные методы с валидацией входных параметров. Реализовал проверку корректности данных: отрицательных сумм, достаточности средств при снятии, обработку некорректного ввода пользователя.

Создал полнофункциональное консольное приложение "Банковский счет", которое демонстрирует все ключевые возможности работы с классами: создание объектов валют, операции зачисления и снятия средств, автоматическую конвертацию между валютами, динамическое отображение баланса в выбранной валюте. Программа корректно обрабатывает ошибочные ситуации и предоставляет понятный интерфейс для пользователя.
Полученные навыки работы с классами, инкапсуляцией и созданием объектов являются фундаментальными для объектно-ориентированного программирования.

Эти знания станут основой для изучения более продвинутых концепций ООП: наследования, полиморфизма, интерфейсов и абстрактных классов, которые будут применяться во всех последующих проектах на C#.